FST – Finite State Transducers

Computation and Discrete Structures III

Juan Marcos Caicedo Mejía — jmcaicedo@icesi.edu.co

Department of Computing and Intelligent Systems
Barberi Faculty of Engineering, Design and Applied Sciences
ICESI University

Material adapted from Professor Oscar Bedoya

2026

Contents

1. Regular Languages and Automata Theory

2. Defining a Finite State Transducer

3. Regular Relations

4. Language of an FST

5. Implementing FST with Pyformlang

6. Exercises

Contents

1. Regular Languages and Automata Theory

2. Defining a Finite State Transducer

3. Regular Relations

4. Language of an FST

5. Implementing FST with Pyformlang

6. Exercises

Where do Transducers fit?

Automata Theory – A Quick Recap

Regular languages are recognized by **Finite State Automata (FSA)**. Transducers extend automata by also *producing* output.

Where do Transducers fit?

Automata Theory – A Quick Recap

Regular languages are recognized by **Finite State Automata (FSA)**. Transducers extend automata by also *producing* output.

Finite State Automaton

- Reads an input string

Where do Transducers fit?

Automata Theory – A Quick Recap

Regular languages are recognized by **Finite State Automata (FSA)**. Transducers extend automata by also *producing* output.

Finite State Automaton

- Reads an input string
- Accepts or rejects it

Where do Transducers fit?

Automata Theory – A Quick Recap

Regular languages are recognized by **Finite State Automata (FSA)**. Transducers extend automata by also *producing* output.

Finite State Automaton

- Reads an input string
- Accepts or rejects it
- One alphabet Σ

Where do Transducers fit?

Automata Theory – A Quick Recap

Regular languages are recognized by **Finite State Automata (FSA)**. Transducers extend automata by also *producing* output.

Finite State Automaton

- Reads an input string
- Accepts or rejects it
- One alphabet Σ
- Output: **boolean**

Where do Transducers fit?

Automata Theory – A Quick Recap

Regular languages are recognized by **Finite State Automata (FSA)**. Transducers extend automata by also *producing* output.

Finite State Automaton

- Reads an input string
- Accepts or rejects it
- One alphabet Σ
- Output: **boolean**

Finite State Transducer

- Reads an input string

Where do Transducers fit?

Automata Theory – A Quick Recap

Regular languages are recognized by **Finite State Automata (FSA)**. Transducers extend automata by also *producing* output.

Finite State Automaton

- Reads an input string
- Accepts or rejects it
- One alphabet Σ
- Output: **boolean**

Finite State Transducer

- Reads an input string
- *Produces* an output string

Where do Transducers fit?

Automata Theory – A Quick Recap

Regular languages are recognized by **Finite State Automata (FSA)**. Transducers extend automata by also *producing* output.

Finite State Automaton

- Reads an input string
- Accepts or rejects it
- One alphabet Σ
- Output: **boolean**

Finite State Transducer

- Reads an input string
- *Produces* an output string
- Two alphabets: Σ (input) and Γ (output)

Where do Transducers fit?

Automata Theory – A Quick Recap

Regular languages are recognized by **Finite State Automata (FSA)**. Transducers extend automata by also *producing* output.

Finite State Automaton

- Reads an input string
- Accepts or rejects it
- One alphabet Σ
- Output: **boolean**

Finite State Transducer

- Reads an input string
- *Produces* an output string
- Two alphabets: Σ (input) and Γ (output)
- Output: **a string** (or a set of strings)

Where do Transducers fit?

Automata Theory – A Quick Recap

Regular languages are recognized by **Finite State Automata (FSA)**. Transducers extend automata by also *producing* output.

Finite State Automaton

- Reads an input string
- Accepts or rejects it
- One alphabet Σ
- Output: **boolean**

Finite State Transducer

- Reads an input string
- *Produces* an output string
- Two alphabets: Σ (input) and Γ (output)
- Output: **a string** (or a set of strings)

An FST computes a *relation* between two languages, not just a membership test.

Contents

1. Regular Languages and Automata Theory

2. Defining a Finite State Transducer

3. Regular Relations

4. Language of an FST

5. Implementing FST with Pyformlang

6. Exercises

What is a Transducer?

Transducer?

An **automaton with output** that transforms input strings into output strings.

What is a Transducer?

Transducer?

An **automaton with output** that transforms input strings into output strings.

- Automata **with output**

What is a Transducer?

Transducer?

An **automaton with output** that transforms input strings into output strings.

- Automata **with output**
- Involves two alphabets: Σ (input) and Γ (output)

What is a Transducer?

Transducer?

An **automaton with output** that transforms input strings into output strings.

- Automata **with output**
- Involves two alphabets: Σ (input) and Γ (output)
- Edges have the form $u : v$ where $u \in \Sigma$ and $v \in \Gamma$, meaning *replace u with v*

Edge notation

An edge labeled $a : b$ means:

read a from input,
write b to output.

ϵ and λ denotes the *empty string* (epsilon).

What is a Transducer?

Transducer?

An **automaton with output** that transforms input strings into output strings.

- Automata **with output**
- Involves two alphabets: Σ (input) and Γ (output)
- Edges have the form $u : v$ where $u \in \Sigma$ and $v \in \Gamma$, meaning *replace u with v*
- Finite transducers define **relations** between inputs and outputs

Edge notation

An edge labeled $a : b$ means:

read a from input,
write b to output.

ϵ and λ denotes the *empty string* (epsilon).

What is a Transducer?

Transducer?

An **automaton with output** that transforms input strings into output strings.

- Automata **with output**
- Involves two alphabets: Σ (input) and Γ (output)
- Edges have the form $u : v$ where $u \in \Sigma$ and $v \in \Gamma$, meaning *replace u with v*
- Finite transducers define **relations** between inputs and outputs
- We can treat finite transductions as **sets of word pairs**

Edge notation

An edge labeled $a : b$ means:

read a from input,
write b to output.

ϵ and λ denotes the *empty string* (epsilon).

Deterministic FST – Formal Definition

A **Deterministic FST** is a 7–tuple $M = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F)$ where:

Deterministic FST – Formal Definition

A **Deterministic FST** is a 7–tuple $M = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F)$ where:

1. Q is the **finite set of states**

Deterministic FST – Formal Definition

A **Deterministic FST** is a 7–tuple $M = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F)$ where:

1. Q is the **finite set of states**
2. Σ is the finite **input alphabet**

Deterministic FST – Formal Definition

A **Deterministic FST** is a 7–tuple $M = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F)$ where:

1. Q is the **finite set of states**
2. Σ is the finite **input alphabet**
3. Γ is the finite **output alphabet**

Deterministic FST – Formal Definition

A **Deterministic FST** is a 7–tuple $M = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F)$ where:

1. Q is the **finite set of states**
2. Σ is the finite **input alphabet**
3. Γ is the finite **output alphabet**
4. $\delta : Q \times (\Sigma \cup \{\lambda\}) \longrightarrow Q$ is the **transition function**

Deterministic FST – Formal Definition

A **Deterministic FST** is a 7–tuple $M = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F)$ where:

1. Q is the **finite set of states**
2. Σ is the finite **input alphabet**
3. Γ is the finite **output alphabet**
4. $\delta : Q \times (\Sigma \cup \{\lambda\}) \longrightarrow Q$ is the **transition function**
5. $\omega : Q \times (\Sigma \cup \{\lambda\}) \longrightarrow \Gamma$ is the **output function**

Deterministic FST – Formal Definition

A **Deterministic FST** is a 7–tuple $M = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F)$ where:

1. Q is the **finite set of states**
2. Σ is the finite **input alphabet**
3. Γ is the finite **output alphabet**
4. $\delta : Q \times (\Sigma \cup \{\lambda\}) \longrightarrow Q$ is the **transition function**
5. $\omega : Q \times (\Sigma \cup \{\lambda\}) \longrightarrow \Gamma$ is the **output function**
6. $q_0 \in Q$ is the **start state**

Deterministic FST – Formal Definition

A **Deterministic FST** is a 7–tuple $M = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F)$ where:

1. Q is the **finite set of states**
2. Σ is the finite **input alphabet**
3. Γ is the finite **output alphabet**
4. $\delta : Q \times (\Sigma \cup \{\lambda\}) \longrightarrow Q$ is the **transition function**
5. $\omega : Q \times (\Sigma \cup \{\lambda\}) \longrightarrow \Gamma$ is the **output function**
6. $q_0 \in Q$ is the **start state**
7. $\emptyset \neq F \subseteq Q$ is the set of **accepting states**

Deterministic FST – Formal Definition

A **Deterministic FST** is a 7–tuple $M = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F)$ where:

1. Q is the **finite set of states**
2. Σ is the finite **input alphabet**
3. Γ is the finite **output alphabet**
4. $\delta : Q \times (\Sigma \cup \{\lambda\}) \longrightarrow Q$ is the **transition function**
5. $\omega : Q \times (\Sigma \cup \{\lambda\}) \longrightarrow \Gamma$ is the **output function**
6. $q_0 \in Q$ is the **start state**
7. $\emptyset \neq F \subseteq Q$ is the set of **accepting states**

Key difference from NFA

δ maps to a *single* state (deterministic). The output function ω maps to a *single* string.

Non-deterministic FST – Formal Definition

A **Non-deterministic FST** is a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F)$ where:

Non-deterministic FST – Formal Definition

A **Non-deterministic FST** is a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F)$ where:

1. Q is the **finite set of states**

Non-deterministic FST – Formal Definition

A **Non-deterministic FST** is a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F)$ where:

1. Q is the **finite set of states**
2. Σ is the finite **input alphabet**

Non-deterministic FST – Formal Definition

A **Non-deterministic FST** is a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F)$ where:

1. Q is the **finite set of states**
2. Σ is the finite **input alphabet**
3. Γ is the finite **output alphabet**

Non-deterministic FST – Formal Definition

A **Non-deterministic FST** is a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F)$ where:

1. Q is the **finite set of states**
2. Σ is the finite **input alphabet**
3. Γ is the finite **output alphabet**
4. $\delta : Q \times (\Sigma \cup \{\lambda\}) \longrightarrow \wp(Q)$ is the **transition function**

Non-deterministic FST – Formal Definition

A **Non-deterministic FST** is a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F)$ where:

1. Q is the **finite set of states**
2. Σ is the finite **input alphabet**
3. Γ is the finite **output alphabet**
4. $\delta : Q \times (\Sigma \cup \{\lambda\}) \longrightarrow \wp(Q)$ is the **transition function**
5. $\omega : Q \times (\Sigma \cup \{\lambda\}) \longrightarrow \Gamma \cup \{\lambda\}$ is the **output function**

Non-deterministic FST – Formal Definition

A **Non-deterministic FST** is a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F)$ where:

1. Q is the **finite set of states**
2. Σ is the finite **input alphabet**
3. Γ is the finite **output alphabet**
4. $\delta : Q \times (\Sigma \cup \{\lambda\}) \longrightarrow \wp(Q)$ is the **transition function**
5. $\omega : Q \times (\Sigma \cup \{\lambda\}) \longrightarrow \Gamma \cup \{\lambda\}$ is the **output function**
6. $q_0 \in Q$ is the **start state**

Non-deterministic FST – Formal Definition

A **Non-deterministic FST** is a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F)$ where:

1. Q is the **finite set of states**
2. Σ is the finite **input alphabet**
3. Γ is the finite **output alphabet**
4. $\delta : Q \times (\Sigma \cup \{\lambda\}) \longrightarrow \wp(Q)$ is the **transition function**
5. $\omega : Q \times (\Sigma \cup \{\lambda\}) \longrightarrow \Gamma \cup \{\lambda\}$ is the **output function**
6. $q_0 \in Q$ is the **start state**
7. $\emptyset \neq F \subseteq Q$ is the set of **accepting states**

Non-deterministic FST – Formal Definition

A **Non-deterministic FST** is a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F)$ where:

1. Q is the **finite set of states**
2. Σ is the finite **input alphabet**
3. Γ is the finite **output alphabet**
4. $\delta : Q \times (\Sigma \cup \{\lambda\}) \longrightarrow \wp(Q)$ is the **transition function**
5. $\omega : Q \times (\Sigma \cup \{\lambda\}) \longrightarrow \Gamma \cup \{\lambda\}$ is the **output function**
6. $q_0 \in Q$ is the **start state**
7. $\emptyset \neq F \subseteq Q$ is the set of **accepting states**

Key difference

δ now maps to a **set of states** $\wp(Q)$ (power set), and ω may produce λ (no output on a transition).

Example 1 – The Identity Transducer

? What does this FST do?

Over the alphabet $\Sigma = \{a, b\}$.

Example 1 – The Identity Transducer

? What does this FST do?

Over the alphabet $\Sigma = \{a, b\}$.

Example 1 – The Identity Transducer

? What does this FST do?

Over the alphabet $\Sigma = \{a, b\}$.

Result

The identity relation.

Every input string is mapped to itself:

$aab \rightarrow aab$
$bba \rightarrow bba$

The simplest possible transducer — it copies input to output unchanged.

Example 2 – Add *a* at the Beginning

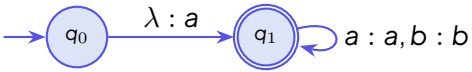
? What does this FST do?

Over the alphabet $\Sigma = \{a, b\}$.

Example 2 – Add *a* at the Beginning

? What does this FST do?

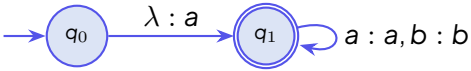
Over the alphabet $\Sigma = \{a, b\}$.



Example 2 – Add *a* at the Beginning

? What does this FST do?

Over the alphabet $\Sigma = \{a, b\}$.



Result

Adds *a* to the beginning of any string.

The $\lambda : a$ transition fires *without consuming input*, outputting *a* immediately.

$bb \rightarrow abb$
$ab \rightarrow aab$

Example 2 – Add *a* at the Beginning

? What does this FST do?

Over the alphabet $\Sigma = \{a, b\}$.

Result

Adds *a* to the beginning of any string.

The $\lambda : a$ transition fires *without consuming input*, outputting *a* immediately.

$bb \rightarrow abb$
$ab \rightarrow aab$

Lambda transitions

λ (epsilon) on the input side means: output without reading any symbol.

Example 3 – Remove Final *b*

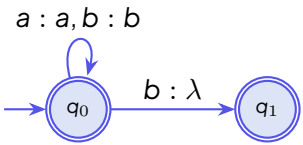
? What does this FST do?

Over the alphabet $\Sigma = \{a, b\}$.

Example 3 – Remove Final *b*

? What does this FST do?

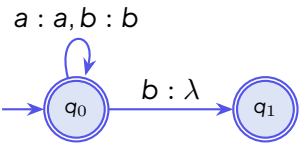
Over the alphabet $\Sigma = \{a, b\}$.



Example 3 – Remove Final *b*

? What does this FST do?

Over the alphabet $\Sigma = \{a, b\}$.



Result

Removes the final *b* if present; rejects strings without a final *b*.

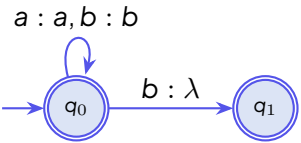
$aab \to aa$
$bab \to ba$

The $b : \lambda$ edge consumes b and produces *nothing*.

Example 3 – Remove Final *b*

? What does this FST do?

Over the alphabet $\Sigma = \{a, b\}$.



Result

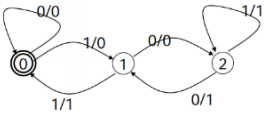
Removes the final *b* if present; rejects strings without a final *b*.

$aab \to aa$
$bab \to ba$

The $b : \lambda$ edge consumes b and produces *nothing*.

Example 3

input	output
0	0
11	01
110	010
1001	0011
1100	0100
1111	0101
10010	00110



Example 4 – Add *b* After Every *a*

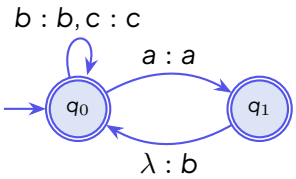
? What does this FST do?

Over the alphabet $\Sigma = \{a, b, c\}$.

Example 4 – Add *b* After Every *a*

? What does this FST do?

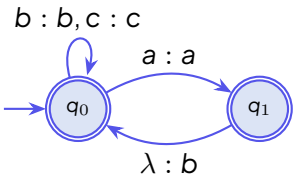
Over the alphabet $\Sigma = \{a, b, c\}$.



Example 4 – Add *b* After Every *a*

? What does this FST do?

Over the alphabet $\Sigma = \{a, b, c\}$.



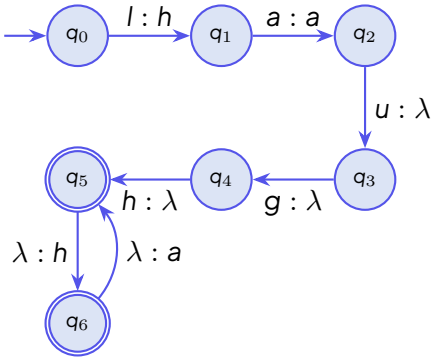
Result

Inserts *b* after every occurrence of *a*.

$ac \rightarrow abc$
$aac \rightarrow ababc$

After reading *a* (and writing *a*), the machine moves to q_1 and immediately outputs *b* via a λ transition.

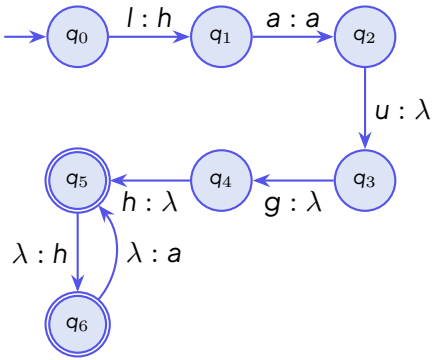
Example 5 – The Laughing Machine



The Laughing Machine

Reads `laugh` and produces `hahaha`...

Example 5 – The Laughing Machine

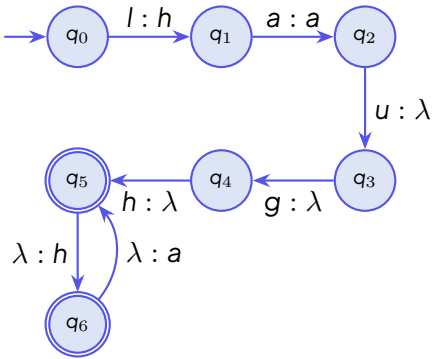


The Laughing Machine

Reads `laugh` and produces `hahaha`...

- $l \rightarrow h$ (reads l, writes h)

Example 5 – The Laughing Machine

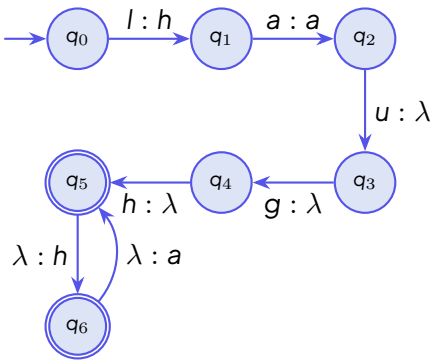


The Laughing Machine

Reads `laugh` and produces `hahaha`...

- $l \rightarrow h$ (reads l, writes h)
- $a \rightarrow a$ (reads a, writes a)

Example 5 – The Laughing Machine

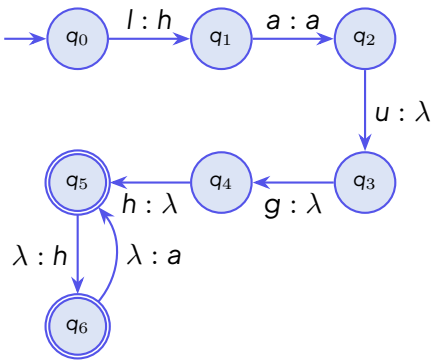


The Laughing Machine

Reads `laugh` and produces `hahaha`...

- $l \to h$ (reads l, writes h)
- $a \to a$ (reads a, writes a)
- $u, g, h \to \lambda$ (consumed silently)

Example 5 – The Laughing Machine

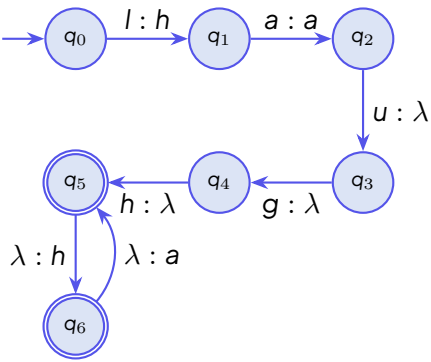


The Laughing Machine

Reads `laugh` and produces `hahaha`...

- $l \to h$ (reads l, writes h)
- $a \to a$ (reads a, writes a)
- $u, g, h \to \lambda$ (consumed silently)
- Then $\lambda \to h$ and $\lambda \to a$ loops produce repeated `ha`

Example 5 – The Laughing Machine



The Laughing Machine

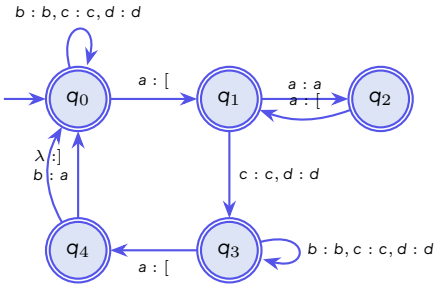
Reads `laugh` and produces `hahaha`...

- $l \to h$ (reads l, writes h)
- $a \to a$ (reads a, writes a)
- $u, g, h \to \lambda$ (consumed silently)
- Then $\lambda \to h$ and $\lambda \to a$ loops produce repeated `ha`

Result

`laugh` $\to$ `hahaha` . . .
A non-deterministic FST with multiple accepting outputs.

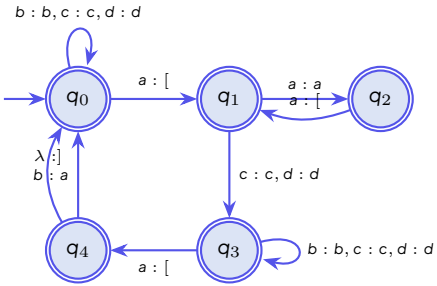
Example 6 – The Bracketing Machine



Bracketing Machine

Every occurrence of *ab* is enclosed in brackets:
$ab \rightarrow [ab]$.

Example 6 – The Bracketing Machine

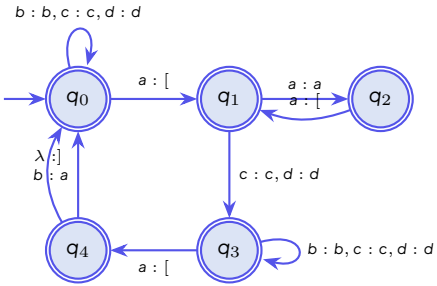


Bracketing Machine

Every occurrence of *ab* is enclosed in brackets: $ab \rightarrow [ab]$.

- When *a* is read, output [(open bracket)

Example 6 – The Bracketing Machine

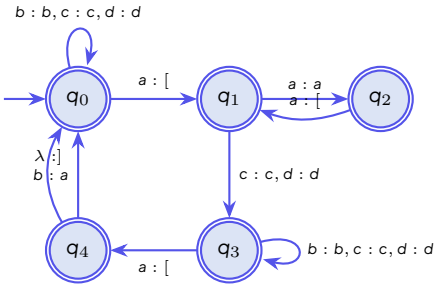


Bracketing Machine

Every occurrence of *ab* is enclosed in brackets: $ab \rightarrow [ab]$.

- When *a* is read, output [(open bracket)
- Then read remaining chars; if *b* follows, output *a* then]

Example 6 – The Bracketing Machine

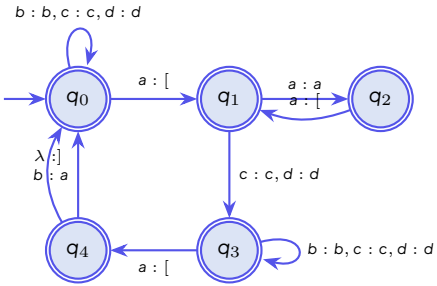


Bracketing Machine

Every occurrence of *ab* is enclosed in brackets: $ab \rightarrow [ab]$.

- When *a* is read, output [(open bracket)
- Then read remaining chars; if *b* follows, output *a* then]
- Other letters pass through unchanged

Example 6 – The Bracketing Machine



Bracketing Machine

Every occurrence of *ab* is enclosed in brackets:
$ab \rightarrow [ab]$.

- When *a* is read, output [(open bracket)
- Then read remaining chars; if *b* follows, output *a* then]
- Other letters pass through unchanged

Examples

$cab \rightarrow c[ab]$
$aabab \rightarrow [a[ab]]$

FST Actions – Computing Relations

What does an FST *compute*?

The action of a Finite State Transducer can be viewed as computing a **relation between two sets**.

FST Actions – Computing Relations

What does an FST *compute*?

The action of a Finite State Transducer can be viewed as computing a **relation between two sets**.

Input side

- A string $x \in \Sigma^*$

FST Actions – Computing Relations

What does an FST *compute*?

The action of a Finite State Transducer can be viewed as computing a **relation between two sets**.

Input side

- A string $x \in \Sigma^*$
- The FST reads x character by character

FST Actions – Computing Relations

What does an FST *compute*?

The action of a Finite State Transducer can be viewed as computing a **relation between two sets**.

Input side

- A string $x \in \Sigma^*$
- The FST reads x character by character
- Transitions determine both next state and output character(s)

FST Actions – Computing Relations

What does an FST *compute*?

The action of a Finite State Transducer can be viewed as computing a **relation between two sets**.

Input side

- A string $x \in \Sigma^*$
- The FST reads x character by character
- Transitions determine both next state and output character(s)

Output side

- A string $y \in \Gamma^*$

FST Actions – Computing Relations

What does an FST *compute*?

The action of a Finite State Transducer can be viewed as computing a **relation between two sets**.

Input side

- A string $x \in \Sigma^*$
- The FST reads x character by character
- Transitions determine both next state and output character(s)

Output side

- A string $y \in \Gamma^*$
- Multiple outputs may exist (non-determinism)

FST Actions – Computing Relations

What does an FST *compute*?

The action of a Finite State Transducer can be viewed as computing a **relation between two sets**.

Input side

- A string $x \in \Sigma^*$
- The FST reads x character by character
- Transitions determine both next state and output character(s)

Output side

- A string $y \in \Gamma^*$
- Multiple outputs may exist (non-determinism)
- The FST defines the *set of pairs* (x, y)

FST Actions – Computing Relations

What does an FST *compute*?

The action of a Finite State Transducer can be viewed as computing a **relation between two sets**.

Input side

- A string $x \in \Sigma^*$
- The FST reads x character by character
- Transitions determine both next state and output character(s)

Output side

- A string $y \in \Gamma^*$
- Multiple outputs may exist (non-determinism)
- The FST defines the *set of pairs* (x, y)

Morphological example

An FST over English words maps each word form to its morphological tag:

`dogs` $\longrightarrow$ `dog+NOUN+pl` `foie` $\longrightarrow$ `foie+NOUN+sg`

Contents

1. Regular Languages and Automata Theory

2. Defining a Finite State Transducer

3. Regular Relations

4. Language of an FST

5. Implementing FST with Pyformlang

6. Exercises

What is a Relation?

Definition

A **relation** over sets A and B is a subset of the Cartesian product $A \times B$.

What is a Relation?

Definition

A **relation** over sets A and B is a subset of the Cartesian product $A \times B$.

Example

Let $A = \{\texttt{dog}, \texttt{cat}, \texttt{cow}\}, \quad B = \{\texttt{seven}, \pi, \texttt{octopus}\}$.

Then $R = \{(\texttt{dog},\ \texttt{seven}),\ (\texttt{cow},\ \pi),\ (\texttt{cow},\ \texttt{octopus})\}$ is a relation.

Note: $\texttt{cow}$ maps to **two** elements — relations allow this, functions do not.

What is a Relation?

Definition

A **relation** over sets A and B is a subset of the Cartesian product $A \times B$.

Example

Let $A = \{\texttt{dog}, \texttt{cat}, \texttt{cow}\}$, $B = \{\texttt{seven}, \pi, \texttt{octopus}\}$.

Then $R = \{(\texttt{dog}, \texttt{seven}), (\texttt{cow}, \pi), (\texttt{cow}, \texttt{octopus})\}$ is a relation.

Note: $\texttt{cow}$ maps to **two** elements — relations allow this, functions do not.

Regular (Rational) Relation

A **regular relation** over Σ, Γ is built from a finite combination of:

1. $\forall (x, y) \in (\Sigma \cup \{\lambda\}) \times (\Gamma \cup \{\lambda\})$
2. $\emptyset$ is a regular relation
3. If R, S are regular relations, so are $R \cdot S$, $R \cup S$, and R^*

Contents

1. Regular Languages and Automata Theory

2. Defining a Finite State Transducer

3. Regular Relations

4. Language of an FST

5. Implementing FST with Pyformlang

6. Exercises

Language of an FST

What is the language of an FST?

The language of an FST is the **set of input–output string pairs** the transducer can recognize and transform — unlike an FSA, which only accepts or rejects strings.

Language of an FST

What is the language of an FST?

The language of an FST is the **set of input–output string pairs** the transducer can recognize and transform — unlike an FSA, which only accepts or rejects strings.

Theorem

For a given finite-state transducer T, the language of T is the set of pairs:

$$\mathcal{L}(T) = \{(x, y) \mid T \text{ translates } x \text{ to } y\}$$

where:

Language of an FST

What is the language of an FST?

The language of an FST is the **set of input–output string pairs** the transducer can recognize and transform — unlike an FSA, which only accepts or rejects strings.

Theorem

For a given finite-state transducer T, the language of T is the set of pairs:

$$\mathcal{L}(T) = \{(x, y) \mid T \text{ translates } x \text{ to } y\}$$

where:

- x is an input string from the input alphabet Σ^*

Language of an FST

What is the language of an FST?

The language of an FST is the **set of input–output string pairs** the transducer can recognize and transform — unlike an FSA, which only accepts or rejects strings.

Theorem

For a given finite-state transducer T, the language of T is the set of pairs:

$$\mathcal{L}(T) = \{(x, y) \mid T \text{ translates } x \text{ to } y\}$$

where:

- x is an input string from the input alphabet Σ^*
- y is an output string from the output alphabet Γ^*

Language of an FST

What is the language of an FST?

The language of an FST is the **set of input–output string pairs** the transducer can recognize and transform — unlike an FSA, which only accepts or rejects strings.

Theorem

For a given finite-state transducer T, the language of T is the set of pairs:

$$\mathcal{L}(T) = \{(x, y) \mid T \text{ translates } x \text{ to } y\}$$

where:

- x is an input string from the input alphabet Σ^*
- y is an output string from the output alphabet Γ^*
- There exists a **valid accepting path** in the FST that reads x and produces y

Language of an FST

What is the language of an FST?

The language of an FST is the **set of input–output string pairs** the transducer can recognize and transform — unlike an FSA, which only accepts or rejects strings.

Theorem

For a given finite-state transducer T, the language of T is the set of pairs:

$$\mathcal{L}(T) = \{(x, y) \mid T \text{ translates } x \text{ to } y\}$$

where:

- x is an input string from the input alphabet Σ^*
- y is an output string from the output alphabet Γ^*
- There exists a **valid accepting path** in the FST that reads x and produces y

Intuition

$\mathcal{L}(T)$ is a *regular relation*. When the FST is deterministic, each x maps to at most one y.

Contents

1. Regular Languages and Automata Theory

2. Defining a Finite State Transducer

3. Regular Relations

4. Language of an FST

5. Implementing FST with Pyformlang

6. Exercises

Building an FST – Example

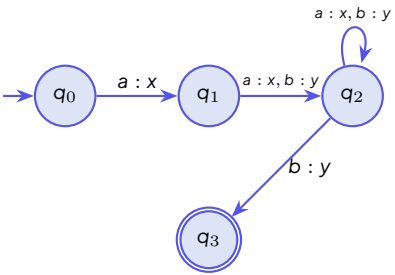
Exercise

Given $\Sigma = \{a, b\}$, build an FST that translates strings of $a(b \cup a)^+ b$ into $x(y \cup x)^+ y$.

Building an FST – Example

Exercise

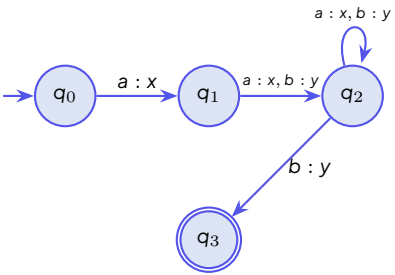
Given $\Sigma = \{a, b\}$, build an FST that translates strings of $a(b \cup a)^+ b$ into $x(y \cup x)^+ y$.



Building an FST – Example

Exercise

Given $\Sigma = \{a, b\}$, build an FST that translates strings of $a(b \cup a)^+ b$ into $x(y \cup x)^+ y$.

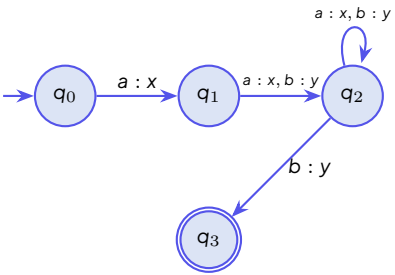


- $q_0 \xrightarrow{a:x} q_1$: read first a, output x

Building an FST – Example

Exercise

Given $\Sigma = \{a, b\}$, build an FST that translates strings of $a(b \cup a)^+b$ into $x(y \cup x)^+y$.

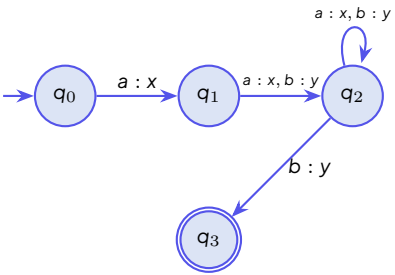


- $q_0 \xrightarrow{a:x} q_1$: read first a, output x
- $q_1 \xrightarrow{a:x/b:y} q_2$: read at least one more char

Building an FST – Example

Exercise

Given $\Sigma = \{a, b\}$, build an FST that translates strings of $a(b \cup a)^+ b$ into $x(y \cup x)^+ y$.

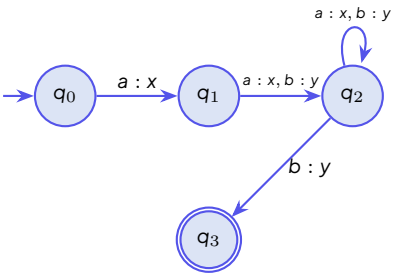


- $q_0 \xrightarrow{a:x} q_1$: read first a, output x
- $q_1 \xrightarrow{a:x/b:y} q_2$: read at least one more char
- q_2 loops: consume the body

Building an FST – Example

Exercise

Given $\Sigma = \{a, b\}$, build an FST that translates strings of $a(b \cup a)^+ b$ into $x(y \cup x)^+ y$.



- $q_0 \xrightarrow{a:x} q_1$: read first a, output x
- $q_1 \xrightarrow{a:x/b:y} q_2$: read at least one more char
- q_2 loops: consume the body
- $q_2 \xrightarrow{b:y} q_3$: final b mapped to y, accept

Pyformlang – Implementation

Python code

```python
from pyformlang.fst import FST

transducer = FST()
transducer.add_transitions([
    ('q0', 'a', 'q1', ['x']),
    ('q1', 'a', 'q2', ['x']),
    ('q1', 'b', 'q2', ['y']),
    ('q2', 'a', 'q2', ['x']),
    ('q2', 'b', 'q2', ['y']),
    ('q2', 'b', 'q3', ['y'])
])
transducer.add_start_state('q0')
transducer.add_final_state('q3')

print("".join(list(transducer.translate('ababb'))[0]))
print("".join(list(transducer.translate('aaaaab'))[0]))
print("".join(list(transducer.translate('abb'))[0]))
```

Pyformlang – Implementation

Python code

```python
from pyformlang.fst import FST

transducer = FST()
transducer.add_transitions([
    ('q0', 'a', 'q1', ['x']),
    ('q1', 'a', 'q2', ['x']),
    ('q1', 'b', 'q2', ['y']),
    ('q2', 'a', 'q2', ['x']),
    ('q2', 'b', 'q2', ['y']),
    ('q2', 'b', 'q3', ['y'])
])
transducer.add_start_state('q0')
transducer.add_final_state('q3')

print("".join(list(transducer.translate('ababb'))[0]))
print("".join(list(transducer.translate('aaaaab'))[0]))
print("".join(list(transducer.translate('abb'))[0]))
```

Verb Conjugation FST – Design

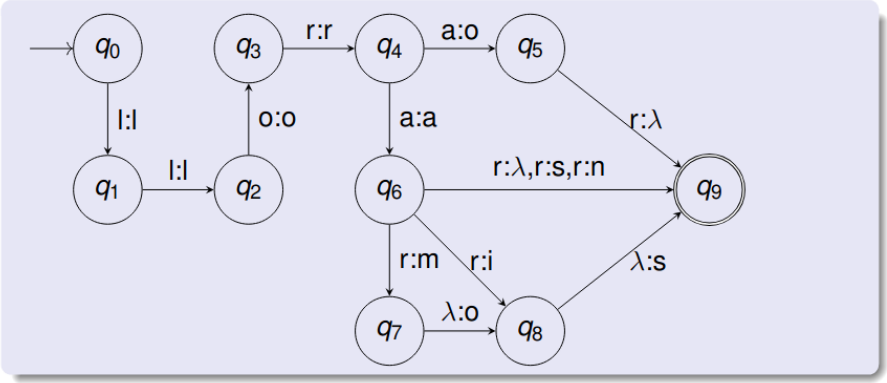
Exercise

Build an FST that, given the verb `llorar` in infinitive, produces all possible conjugations (present tense, indicative).

Verb Conjugation FST – Design

Exercise

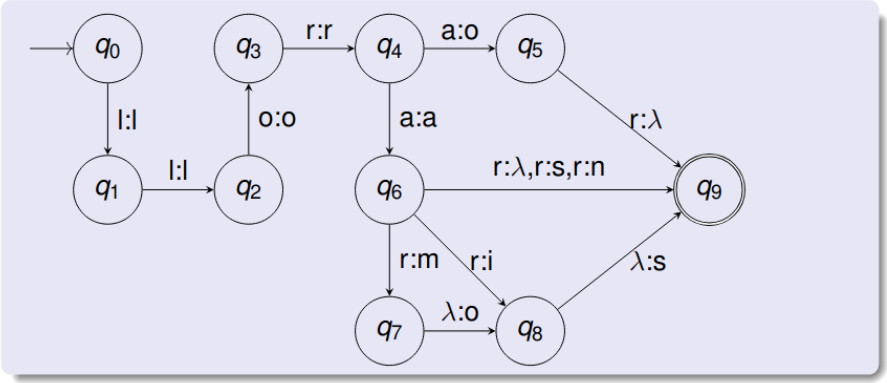
Build an FST that, given the verb `llorar` in infinitive, produces all possible conjugations (present tense, indicative).



Verb Conjugation FST – Design

Exercise

Build an FST that, given the verb `llorar` in infinitive, produces all possible conjugations (present tense, indicative).



The stem `llor` is preserved; the ending `ar` is replaced by each conjugation suffix.

Verb Conjugation – Complete Implementation

Building the Transducer

```
from pyformlang.fst import FST

conjug = FST()
conjug.add_transitions([('q0','l','q1',['l']),
    ('q1','l','q2',['l']), ('q2','o','q3',['o']),
    ('q3','r','q4',['r']), ('q4','a','q5',['o']),
    ('q4','a','q6',['a']), ('q6','r','q7',['m']),
    ('q6','r','q8',['i']), ('q7','epsilon','q8',['o']),
    ('q8','epsilon','q9',['s']), ('q5','r','q9',['']),
    ('q6','r','q9',['']), ('q6','r','q9',['s']),
    ('q6','r','q9',['n'])])
conjug.add_start_state('q0')
conjug.add_final_state('q9')
```

Verb Conjugation – Complete Implementation

Building the Transducer

```
from pyformlang.fst import FST

conjug = FST()
conjug.add_transitions([('q0','l','q1',['l']),
    ('q1','l','q2',['l']), ('q2','o','q3',['o']),
    ('q3','r','q4',['r']), ('q4','a','q5',['o']),
    ('q4','a','q6',['a']), ('q6','r','q7',['m']),
    ('q6','r','q8',['i']), ('q7','epsilon','q8',['o']),
    ('q8','epsilon','q9',['s']), ('q5','r','q9',['']),
    ('q6','r','q9',['']), ('q6','r','q9',['s']),
    ('q6','r','q9',['n'])])
conjug.add_start_state('q0')
conjug.add_final_state('q9')
```

Execution: Processing Output

```
print(list(map(lambda x:
  "".join(x), list(conjug.translate('llorar')))))
```

Understanding Output Processing

What does each function do?

The print statement chains three functions to transform the raw FST output.

Understanding Output Processing

What does each function do?

The print statement chains three functions to transform the raw FST output.

Step 1: `conjug.translate()`

- Reads input string `'llorar'`

Understanding Output Processing

What does each function do?

The print statement chains three functions to transform the raw FST output.

Step 1: `conjug.translate()`

- Reads input string `'llorar'`
- Returns ALL possible paths through the FST

Understanding Output Processing

What does each function do?

The print statement chains three functions to transform the raw FST output.

Step 1: `conjug.translate()`

- Reads input string `'llorar'`
- Returns ALL possible paths through the FST
- Each path = one conjugation output

Understanding Output Processing

What does each function do?

The print statement chains three functions to transform the raw FST output.

Step 1: `conjug.translate()`

- Reads input string `'llorar'`
- Returns ALL possible paths through the FST
- Each path = one conjugation output
- **Output structure:**

```
[['l','l','o','r','o'],
['l','l','o','r','a'],
[…]
```

Understanding Output Processing

What does each function do?

The print statement chains three functions to transform the raw FST output.

Step 1: `conjug.translate()`

- Reads input string `'llorar'`
- Returns ALL possible paths through the FST
- Each path = one conjugation output
- **Output structure:**

```
[['l','l','o','r','o'],
['l','l','o','r','a'],
[…]
```

Understanding Output Processing

What does each function do?

The print statement chains three functions to transform the raw FST output.

Step 1: `conjug.translate()`

- Reads input string `'llorar'`
- Returns ALL possible paths through the FST
- Each path = one conjugation output
- **Output structure:**

```
[['l','l','o','r','o'],
['l','l','o','r','a'],
[...]
```

Why multiple outputs?

- `llorar` is **non-deterministic**

Understanding Output Processing

What does each function do?

The print statement chains three functions to transform the raw FST output.

Step 1: `conjug.translate()`

- Reads input string `'llorar'`
- Returns ALL possible paths through the FST
- Each path = one conjugation output
- **Output structure:**

```
[['l','l','o','r','o'],
['l','l','o','r','a'],
[...]
```

Why multiple outputs?

- `llorar` is **non-deterministic**
- At state q_4, reading a can go to:
  - q_5 output `'o'` (yo lloro)
  - q_6 output `'a'` (tú lloras)
  - etc.

Understanding Output Processing

What does each function do?

The print statement chains three functions to transform the raw FST output.

Step 1: `conjug.translate()`

- Reads input string `'llorar'`
- Returns ALL possible paths through the FST
- Each path = one conjugation output
- **Output structure:**

```
[['l','l','o','r','o'],
['l','l','o','r','a'],
[...]
```

Why multiple outputs?

- `llorar` is **non-deterministic**
- At state q_4, reading a can go to:
  - q_5 output `'o'` (yo lloro)
  - q_6 output `'a'` (tú lloras)
  - etc.
- Each path generates a different conjugation

Understanding Output Processing

What does each function do?

The print statement chains three functions to transform the raw FST output.

Step 1: `conjug.translate()`

- Reads input string `'llorar'`
- Returns ALL possible paths through the FST
- Each path = one conjugation output
- **Output structure:**

```
[['l','l','o','r','o'],
['l','l','o','r','a'],
[...]
```

Why multiple outputs?

- `llorar` is **non-deterministic**
- At state q_4, reading a can go to:
  - q_5 output `'o'` (yo lloro)
  - q_6 output `'a'` (tú lloras)
  - etc.
- Each path generates a different conjugation
- `translate()` finds ALL paths

Step 2 – Wrapping with `list()`

Converting the generator

Step 2 – Wrapping with `list()`

Converting the generator

What `translate()` returns:

- Not a Python `list` but a **generator**

Step 2 – Wrapping with `list()`

Converting the generator

What `translate()` returns:

- Not a Python `list` but a **generator**
- Generators are *lazy* — they compute results on demand

Step 2 – Wrapping with `list()`

Converting the generator

What `translate()` returns:

- Not a Python `list` but a **generator**
- Generators are *lazy* — they compute results on demand
- More memory-efficient for large transducers

Step 2 – Wrapping with `list()`

Converting the generator

What `translate()` returns:

- Not a Python `list` but a **generator**
- Generators are *lazy* — they compute results on demand
- More memory-efficient for large transducers

Why `list()` is needed:

- We need to see ALL outputs at once

Step 2 – Wrapping with `list()`

Converting the generator

What `translate()` returns:

- Not a Python `list` but a **generator**
- Generators are *lazy* — they compute results on demand
- More memory-efficient for large transducers

Why `list()` is needed:

- We need to see ALL outputs at once
- Converts generator **to** eager list

Step 2 – Wrapping with `list()`

Converting the generator

What `translate()` returns:

- Not a Python `list` but a **generator**
- Generators are *lazy* — they compute results on demand
- More memory-efficient for large transducers

Why `list()` is needed:

- We need to see ALL outputs at once
- Converts generator **to** eager list
- After `list()`, we have:

```
[['l','l','o','r','o'], ['l','l','o','r','a'], …]
```

Step 2 – Wrapping with `list()`

Converting the generator

What `translate()` returns:

- Not a Python `list` but a **generator**
- Generators are *lazy* — they compute results on demand
- More memory-efficient for large transducers

Why `list()` is needed:

- We need to see ALL outputs at once
- Converts generator **to** eager list
- After `list()`, we have:

```
[['l','l','o','r','o'], ['l','l','o','r','a'], …]
```

Key insight

Without `list()`, you'd iterate one result at a time. With it, all non-deterministic paths are materialized.

Step 3 – The `lambda` Function

- Anonymous function for transformation

Step 3 – The `lambda` Function

Anonymous function for transformation

What is `lambda x`?

- A **lambda** = anonymous inline function (no `def`)

Step 3 – The `lambda` Function

Anonymous function for transformation

What is `lambda x`?

- A **lambda** = anonymous inline function (no `def`)
- Syntax: `lambda` *`parameter`* `:` *`expression`*

Step 3 – The `lambda` Function

Anonymous function for transformation

What is `lambda x`?

- A **lambda** = anonymous inline function (no `def`)
- Syntax: `lambda` *`parameter`* `:` *`expression`*
- Here: `lambda x : "".join(x)`

Step 3 – The `lambda` Function

Anonymous function for transformation

What is `lambda x`?

- A **lambda** = anonymous inline function (no `def`)
- Syntax: `lambda` *`parameter`* `:` *`expression`*
- Here: `lambda x : "".join(x)`

What does `"".join(x)` do?

- **Input:** a list of characters `x = ['l','l','o','r','o']`

Step 3 – The `lambda` Function

Anonymous function for transformation

What is `lambda x`?

- A **lambda** = anonymous inline function (no `def`)
- Syntax: `lambda` *`parameter`* `:` *`expression`*
- Here: `lambda x : "".join(x)`

What does `"".join(x)` do?

- **Input:** a list of characters `x = ['l','l','o','r','o']`
- **Process:** joins them with empty string separator

Step 3 – The `lambda` Function

Anonymous function for transformation

What is `lambda x`?

- A **lambda** = anonymous inline function (no `def`)
- Syntax: `lambda` *`parameter`* `:` *`expression`*
- Here: `lambda x : "".join(x)`

What does `"".join(x)` do?

- **Input:** a list of characters `x = ['l','l','o','r','o']`
- **Process:** joins them with empty string separator
- **Output:** single string `'lloro'`

Step 3 – The `lambda` Function

Anonymous function for transformation

What is `lambda x`?

- A **lambda** = anonymous inline function (no `def`)
- Syntax: `lambda` *`parameter`* `:` *`expression`*
- Here: `lambda x : "".join(x)`

What does `"".join(x)` do?

- **Input:** a list of characters `x = ['l','l','o','r','o']`
- **Process:** joins them with empty string separator
- **Output:** single string `'lloro'`

Transformation

`['l','l','o','r','a']` → `"".join()` → `'llora'`

Step 4 – Applying with `map()`

- Apply lambda to every output

Step 4 – Applying with `map()`

Apply lambda to every output

What `map()` does

- Applies a *function* to every element in a collection

Step 4 – Applying with `map()`

Apply lambda to every output

What `map()` does

- Applies a *function* to every element in a collection
- Syntax: `map(`*`function, iterable`*`)`

Step 4 – Applying with `map()`

Apply lambda to every output

What `map()` does

- Applies a *function* to every element in a collection
- Syntax: `map(`*`function, iterable`*`)`
- Here: `map(lambda x: "".join(x), list(conjug.translate('llorar')))`

Step 4 – Applying with `map()`

Apply lambda to every output

What `map()` does

- Applies a *function* to every element in a collection
- Syntax: `map(`*`function, iterable`*`)`
- Here: `map(lambda x: "".join(x), list(conjug.translate('llorar')))`

How it flows:

Input (list of outputs)	Lambda function	Result
`['l','l','o','r','o']`	`lambda x: "".join(x)`	`'lloro'`
`['l','l','o','r','a']`	`lambda x: "".join(x)`	`'llora'`
`['l','l','o','r','a','s']`	`lambda x: "".join(x)`	`'lloras'`
⋮	⋮	⋮

Step 4 – Applying with `map()`

Apply lambda to every output

What `map()` does

- Applies a *function* to every element in a collection
- Syntax: `map(`*`function, iterable`*`)`
- Here: `map(lambda x: "".join(x), list(conjug.translate('llorar')))`

How it flows:

Input (list of outputs)	Lambda function	Result
`['l','l','o','r','o']`	`lambda x: "".join(x)`	`'lloro'`
`['l','l','o','r','a']`	`lambda x: "".join(x)`	`'llora'`
`['l','l','o','r','a','s']`	`lambda x: "".join(x)`	`'lloras'`
⋮	⋮	⋮

Result after `map()`

A generator of strings: `'lloro'`, `'llora'`, `'lloras'`, …

Complete Chain – From Input to Final Output

The full transformation pipeline

Why this chain?

The FST explores **all non-deterministic branches**. Each `lambda` unpacks one output path from list-of-lists to string. Both `list()` calls materialize generators into concrete data.

Complete Chain – From Input to Final Output

The full transformation pipeline

Input: `'llorar'`

Why this chain?

The FST explores **all non-deterministic branches**. Each `lambda` unpacks one output path from list-of-lists to string. Both `list()` calls materialize generators into concrete data.

Complete Chain – From Input to Final Output

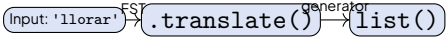
The full transformation pipeline

Why this chain?

The FST explores **all non-deterministic branches**. Each `lambda` unpacks one output path from list-of-lists to string. Both `list()` calls materialize generators into concrete data.

Complete Chain – From Input to Final Output

The full transformation pipeline

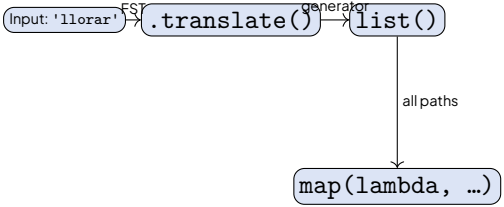


Why this chain?

The FST explores **all non-deterministic branches**. Each `lambda` unpacks one output path from list-of-lists to string. Both `list()` calls materialize generators into concrete data.

Complete Chain – From Input to Final Output

The full transformation pipeline

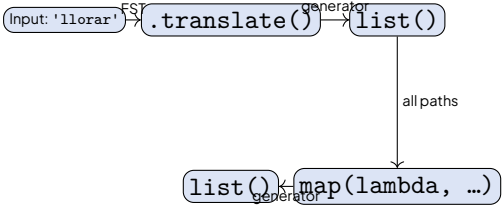


Why this chain?

The FST explores **all non-deterministic branches**. Each `lambda` unpacks one output path from list-of-lists to string. Both `list()` calls materialize generators into concrete data.

Complete Chain – From Input to Final Output

The full transformation pipeline

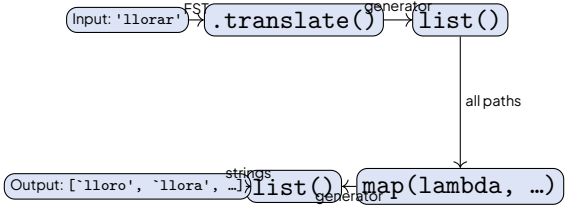


Why this chain?

The FST explores **all non-deterministic branches**. Each `lambda` unpacks one output path from list-of-lists to string. Both `list()` calls materialize generators into concrete data.

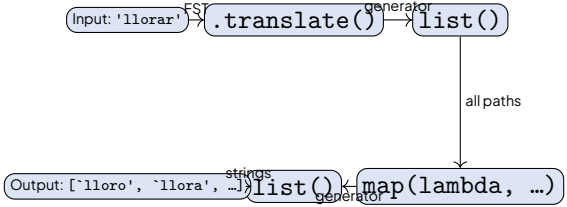
Complete Chain – From Input to Final Output

The full transformation pipeline



Complete Chain – From Input to Final Output

The full transformation pipeline



Why this chain?

The FST explores **all non-deterministic branches**. Each `lambda` unpacks one output path from list-of-lists to string. Both `list()` calls materialize generators into concrete data.

Why Multiple Outputs?

Non-determinism in the FST

The transducer has multiple valid accepting paths for the same input.

Why Multiple Outputs?

Non-determinism in the FST

The transducer has multiple valid accepting paths for the same input.

Different paths for `'llorar'`

- $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow q_4 \xrightarrow{a:o} q_5 \xrightarrow{r:\lambda} q_9 \Rightarrow$ `lloro`

Why Multiple Outputs?

Non-determinism in the FST

The transducer has multiple valid accepting paths for the same input.

Different paths for `'llorar'`

- $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow q_4 \xrightarrow{a:o} q_5 \xrightarrow{r:\lambda} q_9 \Rightarrow$ `lloro`
- $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow q_4 \xrightarrow{a:a} q_6 \xrightarrow{r:\lambda} q_9 \Rightarrow$ `llora`

Why Multiple Outputs?

Non-determinism in the FST

The transducer has multiple valid accepting paths for the same input.

Different paths for `'llorar'`

- $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow q_4 \xrightarrow{a:o} q_5 \xrightarrow{r:\lambda} q_9 \Rightarrow$ `lloro`
- $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow q_4 \xrightarrow{a:a} q_6 \xrightarrow{r:\lambda} q_9 \Rightarrow$ `llora`
- $q_0 \rightarrow \cdots \rightarrow q_4 \xrightarrow{a:a} q_6 \xrightarrow{r:m} q_7 \xrightarrow{\epsilon:o} q_8 \xrightarrow{\epsilon:s} q_9 \Rightarrow$ `lloramos`

Why Multiple Outputs?

Non-determinism in the FST

The transducer has multiple valid accepting paths for the same input.

Different paths for `'llorar'`

- $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow q_4 \xrightarrow{a:o} q_5 \xrightarrow{r:\lambda} q_9 \Rightarrow$ `lloro`
- $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow q_4 \xrightarrow{a:a} q_6 \xrightarrow{r:\lambda} q_9 \Rightarrow$ `llora`
- $q_0 \rightarrow \cdots \rightarrow q_4 \xrightarrow{a:a} q_6 \xrightarrow{r:m} q_7 \xrightarrow{\epsilon:o} q_8 \xrightarrow{\epsilon:s} q_9 \Rightarrow$ `lloramos`
- And so on...

Why Multiple Outputs?

Non-determinism in the FST

The transducer has multiple valid accepting paths for the same input.

Different paths for `'llorar'`

- $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow q_4 \xrightarrow{a:o} q_5 \xrightarrow{r:\lambda} q_9 \Rightarrow$ `lloro`
- $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow q_4 \xrightarrow{a:a} q_6 \xrightarrow{r:\lambda} q_9 \Rightarrow$ `llora`
- $q_0 \rightarrow \cdots \rightarrow q_4 \xrightarrow{a:a} q_6 \xrightarrow{r:m} q_7 \xrightarrow{\epsilon:o} q_8 \xrightarrow{\epsilon:s} q_9 \Rightarrow$ `lloramos`
- And so on...

Why?

- At q_4, the input symbol a can transition to:
  - q_5 (output o)
  - q_6 (output a)

Why Multiple Outputs?

Non-determinism in the FST

The transducer has multiple valid accepting paths for the same input.

Different paths for `'llorar'`

- $q_0 \to q_1 \to q_2 \to q_3 \to q_4 \xrightarrow{a:o} q_5 \xrightarrow{r:\lambda} q_9 \Rightarrow$ `lloro`
- $q_0 \to q_1 \to q_2 \to q_3 \to q_4 \xrightarrow{a:a} q_6 \xrightarrow{r:\lambda} q_9 \Rightarrow$ `llora`
- $q_0 \to \cdots \to q_4 \xrightarrow{a:a} q_6 \xrightarrow{r:m} q_7 \xrightarrow{\epsilon:o} q_8 \xrightarrow{\epsilon:s} q_9 \Rightarrow$ `lloramos`
- And so on...

Why?

- At q_4, the input symbol `a` can transition to:
  - q_5 (output `o`)
  - q_6 (output `a`)
- From q_6, the symbol `r` can go to:
  - q_7 (output `m`)
  - q_8 (output `i`)
  - q_9 (output λ)

Why Multiple Outputs?

Non-determinism in the FST

The transducer has multiple valid accepting paths for the same input.

Different paths for `'llorar'`

- $q_0 \to q_1 \to q_2 \to q_3 \to q_4 \xrightarrow{a:o} q_5 \xrightarrow{r:\lambda} q_9 \Rightarrow$ `lloro`
- $q_0 \to q_1 \to q_2 \to q_3 \to q_4 \xrightarrow{a:a} q_6 \xrightarrow{r:\lambda} q_9 \Rightarrow$ `llora`
- $q_0 \to \cdots \to q_4 \xrightarrow{a:a} q_6 \xrightarrow{r:m} q_7 \xrightarrow{\epsilon:o} q_8 \xrightarrow{\epsilon:s} q_9 \Rightarrow$ `lloramos`
- And so on...

Why?

- At q_4, the input symbol `a` can transition to:
  - q_5 (output `o`)
  - q_6 (output `a`)
- From q_6, the symbol `r` can go to:
  - q_7 (output `m`)
  - q_8 (output `i`)
  - q_9 (output λ)
- Each choice = different conjugation

Why Multiple Outputs?

Non-determinism in the FST

The transducer has multiple valid accepting paths for the same input.

Different paths for `'llorar'`

- $q_0 \to q_1 \to q_2 \to q_3 \to q_4 \xrightarrow{a:o} q_5 \xrightarrow{r:\lambda} q_9 \Rightarrow$ `lloro`
- $q_0 \to q_1 \to q_2 \to q_3 \to q_4 \xrightarrow{a:a} q_6 \xrightarrow{r:\lambda} q_9 \Rightarrow$ `llora`
- $q_0 \to \cdots \to q_4 \xrightarrow{a:a} q_6 \xrightarrow{r:m} q_7 \xrightarrow{\epsilon:o} q_8 \xrightarrow{\epsilon:s} q_9 \Rightarrow$ `lloramos`
- And so on...

Why?

- At q_4, the input symbol `a` can transition to:
  - q_5 (output `o`)
  - q_6 (output `a`)
- From q_6, the symbol `r` can go to:
  - q_7 (output `m`)
  - q_8 (output `i`)
  - q_9 (output λ)
- Each choice = different conjugation

Non-determinism allows one FST to encode all conjugations at once!

Actual Output

Running the complete code

```
['lloro', 'llora', 'lloras', 'lloramos', 'lloran', 'lloráis']
```

Actual Output

Running the complete code

```
['lloro', 'llora', 'lloras', 'lloramos', 'lloran', 'lloráis']
```

Mapping to persons:

- `lloro` = yo lloro (I cry)

Actual Output

Running the complete code

```
['lloro', 'llora', 'lloras', 'lloramos', 'lloran', 'lloráis']
```

Mapping to persons:

- `lloro` = yo lloro (I cry)
- `llora` = él/ella llora (he/she cries)

Actual Output

Running the complete code

```
['lloro', 'llora', 'lloras', 'lloramos', 'lloran', 'lloráis']
```

Mapping to persons:

- `lloro` = yo lloro (I cry)
- `llora` = él/ella llora (he/she cries)
- `lloras` = tú lloras (you cry)

Actual Output

Running the complete code

```
['lloro', 'llora', 'lloras', 'lloramos', 'lloran', 'lloráis']
```

Mapping to persons:

- `lloro` = yo lloro (I cry)
- `llora` = él/ella llora (he/she cries)
- `lloras` = tú lloras (you cry)
- `lloramos` = nosotros lloramos (we cry)

Actual Output

Running the complete code

```
['lloro', 'llora', 'lloras', 'lloramos', 'lloran', 'lloráis']
```

Mapping to persons:

- `lloro` = yo lloro (I cry)
- `llora` = él/ella llora (he/she cries)
- `lloras` = tú lloras (you cry)
- `lloramos` = nosotros lloramos (we cry)
- `lloran` = ellos/ellas lloran (they cry)

Actual Output

Running the complete code

```
['lloro', 'llora', 'lloras', 'lloramos', 'lloran', 'lloráis']
```

Mapping to persons:

- `lloro` = yo lloro (I cry)
- `llora` = él/ella llora (he/she cries)
- `lloras` = tú lloras (you cry)
- `lloramos` = nosotros lloramos (we cry)
- `lloran` = ellos/ellas lloran (they cry)
- `lloráis` = vosotros lloráis (you all cry [Spain])

Actual Output

Running the complete code

```
['lloro', 'llora', 'lloras', 'lloramos', 'lloran', 'lloráis']
```

Mapping to persons:

- `lloro` = yo lloro (I cry)
- `llora` = él/ella llora (he/she cries)
- `lloras` = tú lloras (you cry)
- `lloramos` = nosotros lloramos (we cry)
- `lloran` = ellos/ellas lloran (they cry)
- `lloráis` = vosotros lloráis (you all cry [Spain])

Key takeaway

A single FST input generates **all 6 conjugations** by exploring different non-deterministic paths.

The processing chain unpacks these efficiently!

Verb Conjugation – Pyformlang (Compact Ve

Simplified implementation

Uses multi-character transitions for conciseness.

```
from pyformlang.fst import FST

conjug = FST()
conjug.add_transitions([
    ('q0', 'llor', 'q1', ['llor']),
    ('q1', 'ar',   'q2', ['o'   ]),
    ('q1', 'ar',   'q2', ['as'  ]),
    ('q1', 'ar',   'q2', ['a'   ]),
    ('q1', 'ar',   'q2', ['amos']),
    ('q1', 'ar',   'q2', ['an'  ]),
    ('q1', 'ar',   'q2', ['ais' ]),
])
conjug.add_start_state('q0')
conjug.add_final_state('q2')

results = list(map(lambda x: "".join(x),
    list(conjug.translate(['llor', 'ar']))))
```

Verb Conjugation – Pyformlang (Compact Ve

Simplified implementation

Uses multi-character transitions for conciseness.

```
from pyformlang.fst import FST

conjug = FST()
conjug.add_transitions([
    ('q0', 'llor', 'q1', ['llor']),
    ('q1', 'ar',   'q2', ['o'   ]),
    ('q1', 'ar',   'q2', ['as'  ]),
    ('q1', 'ar',   'q2', ['a'   ]),
    ('q1', 'ar',   'q2', ['amos']),
    ('q1', 'ar',   'q2', ['an'  ]),
    ('q1', 'ar',   'q2', ['ais' ]),
])
conjug.add_start_state('q0')
conjug.add_final_state('q2')

results = list(map(lambda x: "".join(x),
    list(conjug.translate(['llor', 'ar']))))
```

Contents

1. Regular Languages and Automata Theory

2. Defining a Finite State Transducer

3. Regular Relations

4. Language of an FST

5. Implementing FST with Pyformlang

6. Exercises

Exercises

Practice

Apply what you have learned to design your own FSTs.

Exercises

Practice

Apply what you have learned to design your own FSTs.

1. Design an FST over $\Sigma = \{a, b\}$ that **reverses** every string (i.e., maps *abba* $\rightarrow$ *abba* and *ab* $\rightarrow$ *ba*). *Hint: is this possible with a finite transducer?*

Exercises

Practice

Apply what you have learned to design your own FSTs.

1. Design an FST over $\Sigma = \{a, b\}$ that **reverses** every string (i.e., maps *abba* $\rightarrow$ *abba* and *ab* $\rightarrow$ *ba*). *Hint: is this possible with a finite transducer?*
2. Design an FST that maps every string $w \in \{0, 1\}^*$ to w with all 0s replaced by 00 and all 1s unchanged.

Exercises

Practice

Apply what you have learned to design your own FSTs.

1. Design an FST over $\Sigma = \{a, b\}$ that **reverses** every string (i.e., maps *abba* $\rightarrow$ *abba* and *ab* $\rightarrow$ *ba*). *Hint: is this possible with a finite transducer?*
2. Design an FST that maps every string $w \in \{0, 1\}^*$ to w with all 0s replaced by 00 and all 1s unchanged.
3. Build a Pyformlang FST for the verb `cantar` (to sing) and produce all present indicative conjugations.

Exercises

Practice

Apply what you have learned to design your own FSTs.

1. Design an FST over $\Sigma = \{a, b\}$ that **reverses** every string (i.e., maps *abba* $\rightarrow$ *abba* and *ab* $\rightarrow$ *ba*). *Hint: is this possible with a finite transducer?*
2. Design an FST that maps every string $w \in \{0, 1\}^*$ to w with all 0s replaced by 00 and all 1s unchanged.
3. Build a Pyformlang FST for the verb `cantar` (to sing) and produce all present indicative conjugations.
4. Design an FST that recognizes strings of the form $a^n b^n$ and maps them to $x^n y^n$. Is this possible with a finite transducer? *Justify your answer.*

Exercises

Practice

Apply what you have learned to design your own FSTs.

1. Design an FST over $\Sigma = \{a, b\}$ that **reverses** every string (i.e., maps *abba* $\rightarrow$ *abba* and *ab* $\rightarrow$ *ba*). *Hint: is this possible with a finite transducer?*
2. Design an FST that maps every string $w \in \{0, 1\}^*$ to w with all 0s replaced by 00 and all 1s unchanged.
3. Build a Pyformlang FST for the verb `cantar` (to sing) and produce all present indicative conjugations.
4. Design an FST that recognizes strings of the form $a^n b^n$ and maps them to $x^n y^n$. Is this possible with a finite transducer? *Justify your answer.*

Discuss your designs with a classmate: can you always determine *uniquely* what an FST does from its diagram?